Report for Assignment 4

Amanda Zakir
Dawa Arkhang
Edvin Livak
Jafar Mohammadi
Yusuf Can Ekin

March 2, 2026

Contents

1 Project **3**

1.1 Architecture and Purpose of the System . . . 3

1.1.1 Purpose of the System . . . 3

1.1.2 High-Level Architecture . . . 3

1.1.3 Execution Flow . . . 3

1.2 Architectural Relevance of the Selected Issue . . . 4

1.3 Architectural Characteristics . . . 4

2 Onboarding Experience **6**

3 Effort Spent **7**

4 Overview of Issue(s) and Work Done **9**

4.1 Technical Investigation: JavaScript Context Persistence . . . 9

4.2 Reproducing the Issue and Test Structure . . . 10

5 Requirements for the New Feature or Requirements Affected by Functionality Being Refactored **12**

5.1 Problem Statement . . . 12

5.2 Requirements: . . . 13

5.3 Identified scenarios . . . 15

5.4 Trace tests to requirements . . . 16

6 Code Changes **16**

6.1 Patch . . . 16

7 Test Log Results **17**

8 UML Class Diagram Before Refactoring **20**

9 UML Class Diagram After Refactoring **21**

10 Overall Experience **22**

10.1 Main Takeaways 22

10.2 Critical Reflection in the Context of Software Engineering Practice and the SEMAT Kernel 22

10.2.1 Self Assessment: Way of Working 23

10.2.2 Context Within Software Engineering Practices 24

10.3 Additional Reflections 24

1 Project

Name: Karate Labs
URL: `https://github.com/karatelabs/karate`

1.1 Architecture and Purpose of the System

1.1.1 Purpose of the System

Karate is an open-source testing framework designed for API testing, automation, and behavior-driven development (BDD). It enables users to write test scenarios in a domain-specific language that combines Gherkin-style syntax with embedded JavaScript expressions. The primary goal of the system is to simplify API testing by reducing boilerplate code and integrating assertions, HTTP calls, and data handling into a unified framework.

Unlike traditional Java testing libraries such as JUnit, Karate provides a higher-level abstraction tailored specifically for HTTP-based services. It allows users to define reusable features, call other feature files, and share variables across execution contexts. This makes the system both powerful and architecturally non-trivial.

1.1.2 High-Level Architecture

At a high level, Karate consists of the following main subsystems:

- **Parser Layer:** Responsible for reading and parsing feature files written in Gherkin syntax.
- **Execution Engine:** Orchestrates the execution of scenarios and steps.
- **Runtime Context Management:** Maintains variable scopes, configuration, and state across nested feature calls.
- **JavaScript Integration Layer:** Executes embedded JavaScript expressions and manages interaction between Java and the scripting engine.
- **HTTP Client Layer:** Handles outgoing API requests and response processing.

The core execution logic resides primarily in the `karate-core` module. The execution engine coordinates scenario evaluation, delegates JavaScript expressions to the embedded engine, and ensures that variables and state are correctly propagated between features.

1.1.3 Execution Flow

When a feature file is executed, the system performs the following steps:

1. Parse the feature file into an internal representation.
2. Initialize a runtime context containing configuration and global variables.
3. Execute each scenario step sequentially.

4. Evaluate embedded JavaScript expressions when encountered.
5. Manage nested feature calls while preserving or isolating variable scope.

The interaction between the execution engine and the JavaScript engine is particularly important. Java objects and variables are exposed to JavaScript, and modifications in JavaScript may affect the runtime context. This interaction requires careful state management.

1.2 Architectural Relevance of the Selected Issue

The issue we selected relates to the handling of global variables across nested feature executions. Specifically, it concerns how the runtime context is preserved or restored when a feature invokes another feature.

Architecturally, this affects:

- The execution engine
- Runtime context management
- JavaScript variable handling

Because the issue involves state propagation across execution boundaries, it impacts multiple classes and cannot be resolved within a single isolated function. This makes it architecturally significant rather than a local bug fix.

1.3 Architectural Characteristics

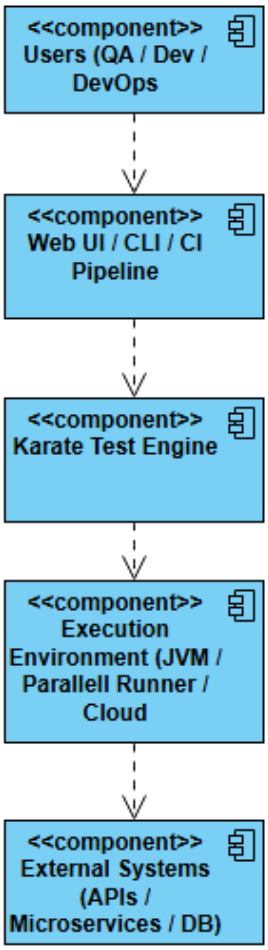
Karate follows a layered architecture with clear separation between parsing, execution, and external communication (HTTP). However, the runtime context management introduces cross-cutting concerns, particularly regarding variable scope and state consistency.

The design reflects characteristics of:

- Interpreter pattern (for executing feature steps)
- Facade pattern (simplifying complex API testing logic)
- Context object pattern (for runtime state handling)

Understanding these architectural elements was essential in order to reason about the selected issue and propose a robust solution.

Architectural Overview Diagram



At the top layer, users (QA engineers, developers, and DevOps engineers) interact with the system through a Web UI, Command Line Interface (CLI), or CI/CD pipeline. These interfaces act as entry points that trigger the execution of feature files.

The central component in the diagram, the *Karate Test Engine*, corresponds to the internal subsystems described earlier. It encapsulates the parser layer, execution engine, runtime context management, JavaScript integration layer, and HTTP client layer. This component is responsible for interpreting feature files and orchestrating test execution.

The execution environment layer represents the runtime infrastructure in which the engine operates. This includes the JVM, support for parallel execution, and potential cloud-based execution contexts. While not part of the logical core, this layer is architecturally significant because it determines scalability and execution isolation.

Finally, the system under test (external APIs, microservices, and databases) represents the external boundary of the system. Karate communicates with these systems through its HTTP client layer, validating responses according to defined test scenarios.

The diagram therefore complements the internal architectural breakdown by showing how the core execution logic is embedded within a broader layered system context.

2 Onboarding Experience

For this assignment we chose a new project: the open-source Java testing framework *Karate*. The project is maintained on GitHub and is actively developed, with a large codebase and extensive documentation.

In previous assignments, we worked with the Apache Commons Lang project, which is a general-purpose Java utility library. Compared to that experience, onboarding to Karate was noticeably different.

Apache Commons Lang is a mature and relatively self-contained utility library. Its architecture is modular but conceptually simple, primarily consisting of independent helper classes (e.g., for string manipulation, object utilities, and number handling). Understanding individual issues often required examining isolated methods within a single class, and most changes were localized.

In contrast, Karate is a full-featured testing framework with a significantly more complex execution architecture. The project integrates Java with a JavaScript engine and supports nested feature execution, runtime contexts, and automated test orchestration. As a result, understanding our selected issue required tracing execution flow across multiple classes and modules rather than modifying a single utility function.

Another key difference was the nature of the system. Apache Commons Lang is primarily a static library without runtime orchestration, whereas Karate contains an execution engine that manages feature parsing, scenario execution, and JavaScript evaluation. This increased architectural complexity made the onboarding phase more demanding but also more educational, particularly in terms of understanding framework-level design and context management.

Overall, while Apache Commons Lang was easier to grasp at a local code level, Karate provided deeper insight into software architecture, runtime behavior, and integration between multiple technologies. If you changed the project, how did your experience differ from before?

2.1 Build Experience and Tooling

Overall, the project was relatively straightforward to build and run. The repository includes a well-structured README file with clear written instructions explaining the setup process. In addition, the documentation contains links to YouTube videos demonstrating installation and usage, which significantly lowers the barrier for new contributors.

(a) **Additional tools required:** The primary tools required were a Java Development Kit (JDK 17 in our case) and Apache Maven. No additional complex frameworks or platform-specific tools were necessary. Since these are standard tools for Java development, the setup process was manageable.

(b) **Documentation of required tools:** The required tools were clearly documented in the README file. The setup instructions included both textual explanations and external resources (including video tutorials), which made the onboarding process accessible and easy to follow.

(c) **Automatic installation of components:** All project dependencies were handled automatically by Maven. Upon running `mvn clean install`, Maven downloaded the required libraries and plugins without manual intervention. This significantly simplified the build process.

(d) **Build stability:** The build completed successfully without errors on our systems. The initial build took slightly longer due to dependency downloads, but subsequent builds were faster due to local caching.

(e) **Examples and test execution:** The provided examples and automated regression tests executed correctly using `mvn clean test`. All tests passed on the original version of the project, confirming that the repository was stable and in a working state before modifications.

In summary, the build process was smooth and well-documented. The combination of clear written instructions, automated dependency management via Maven, and supplementary video material contributed to a positive onboarding experience.

3 Effort Spent

Member 1: Amanda

1. Plenary discussions / meetings: 2.5hrs
2. Discussions within parts of the group: 2hr
3. Reading documentation: 1hr
4. Configuration and setup: 2hr
5. Analyzing code / output: 1hr
6. Writing documentation: 6hr
7. Writing code:
8. Running code: 30mins

Member 2: Dawa

1. Plenary discussions / meeting: 1,5 hrs
2. Discussions within parts of the group: 2 hrs
3. Reading documentation: 5 hrs
4. Configuration and setup: 1 hrs
5. Analyzing code / output: 8 hrs
6. Writing documentation: 5 hrs
7. Writing code:
8. Running code: 40 min

Member 3: Edvin

1. Plenary discussions / meetings: 1.5 hrs
2. Discussions within parts of the group: 0.5 hrs
3. Reading documentation: 2 hrs
4. Configuration and setup: 1 hr
5. Analyzing code / output: 11 hrs
6. Writing documentation: 2 hrs
7. Writing code: 2 hrs
8. Running code: 1 hr

Member 4: Jafar

1. Plenary discussions / meetings: 2.5 hrs
2. Discussions within parts of the group: 2 hrs
3. Reading documentation: 2.5 hrs
4. Configuration and setup: 1 hr
5. Analyzing code / output: 6 hrs
6. Writing documentation: 3.5 hrs
7. Writing code: 5 hrs
8. Running code: 2 hrs

Member 5: Yusuf

1. Plenary discussions / meetings: 2,5 hrs
2. Discussions within parts of the group: 2 hrs
3. Reading documentation: 2 hrs
4. Configuration and setup: 1 hr
5. Analyzing code / output: 5 hrs
6. Writing documentation: 3 hrs
7. Writing code: 2 hrs
8. Running code: 30 mins

4 Overview of Issue(s) and Work Done

Title: JavaScript functions cannot access global vars after executing feature in nested context

URL: `https://github.com/karatelabs/karate/issues/2502`

When a Karate feature calls a Java method that executes another nested Karate feature, JavaScript functions in the parent feature can no longer access global variables, resulting in a PolyglotException, even though Karate steps themselves can still access them. This issue occurs due to the JS/Polyglot context being reset or isolated during nested feature execution.

Scope

The investigation focused on the `karate-core` module, specifically the `com.intuit.karate.Runner`, `com.intuit.karate.core.ScenarioRuntime`, and `com.intuit.karate.core.ScenarioEngine` classes. These classes form the heart of the framework where the execution engine, the JavaScript (GraalVM) integration, and the Runner logic reside.

4.1 Technical Investigation: JavaScript Context Persistence

sub issue: Refactor and analyze JS context lifecycle during nested feature calls
Issue Link: GitHub Issue #17

Root Cause Explanation

The investigation into the JavaScript context creation lifecycle revealed that the loss of global variables is caused by **architectural isolation** during nested execution. When a parent feature triggers a Java utility, such as `KarateBasedJavaUtil`, which subsequently calls a static runner, the framework initiates a top-down re-initialization of the entire execution stack.

The Role of the Suite

The `Suite` acts as the root orchestrator and global "container" for a test run, holding the thread pool, configuration, and, most importantly, the `callOnceCache`. In a standard internal call, the `Suite` is shared, however, the static entry point in `Runner.runFeature()` (or `KarateRunner.executeScenario()`) explicitly executes `new Suite()`. By creating a new `Suite` instance, the framework spawns an entirely independent environment that is "blind" to the parent execution thread's memory.

Method Interconnectivity and Execution Flow

The failure to preserve JavaScript globals is a direct result of how these methods are chained during a nested call:

1. **Runner.runFeature() → FeatureRuntime.of():** The static runner creates the new `Suite` and passes it to the `FeatureRuntime`. Because this is a new suite, it initializes a default `ScenarioCall` object for the `caller`.

2. **FeatureRuntime → ScenarioRuntime Constructor:** When the `ScenarioRuntime` is instantiated, it checks the caller via `featureRuntime.caller`. Since the `Runner` started a fresh suite, `caller.isNone()` evaluates to `true`.
3. **ScenarioRuntime → ScenarioEngine Constructor:** Because `caller.isNone()` is true, the runtime executes `new ScenarioEngine(..., new HashMap<>(), ...)`. The passing of a `new HashMap<>()` instead of the parent's variable map is the specific point of state loss.
4. **ScenarioEngine.init():** Finally, this method is called to initialize the `JS` context. It executes `JS = JsEngine.local()` and populates it only with the empty map provided in the previous step, resulting in a clean-slate GraalVM Polyglot Context.

This chain confirms that the `Suite` initialization is the catalyst that forces the `ScenarioRuntime` to orphan the nested execution, leading to an isolated `ScenarioEngine` that lacks the parent's JavaScript globals.

Primary Code References

The following classes and methods were identified as the specific points where the context resets:

- **com.intuit.karate.Runner [runFeature()]:** The static entry point that instantiates a `new Suite()`, effectively orphaning the nested call from the parent state.
- **com.intuit.karate.core.ScenarioRuntime [Constructor]:** Specifically where the check `if (caller.isNone())` forces the creation of a new, empty `ScenarioEngine`.
- **com.intuit.karate.core.ScenarioEngine [init()]:** The method responsible for spawning the `JsEngine` (Polyglot Context) using the empty variable map provided by the runtime.

Technical Summary

The "Scope Isolation Behavior" intended to protect test integrity currently prevents legitimate state propagation during nested calls.

- **Trace Result:** The JS context is created within `ScenarioEngine.init()` but is limited to variables found in its local `vars` map.
- **Reset Point:** The reset occurs during the instantiation of the nested `ScenarioRuntime`, which fails to "bridge" to the parent engine.
- **Proposed Solution:** To resolve this, the `Runner` must be refactored to allow passing an existing `ScenarioEngine` or `Suite` context into nested calls, ensuring the `caller` state is correctly identified as a child rather than a new root.

4.2 Reproducing the Issue and Test Structure

Pull Request: `https://github.com/yusufcanekin/karate/pull/20`

Reproducing the Issue

The issue can be reproduced by constructing a scenario where a parent Karate feature defines a global JavaScript variable, then delegates execution to a Java utility class that internally invokes a nested Karate feature via the static `Runner.runFeature()` entry point. Upon returning to the parent feature, any JavaScript function that attempts to read the previously defined global variable throws a `PolyglotException`, indicating that the GraalVM context has been reset and the variable binding is no longer accessible.

The minimal reproduction steps are as follows:

1. Define a global variable in the parent feature using a `karate.set()` call or a direct JavaScript assignment.
2. Call a Java method (e.g., `KarateBasedJavaUtil.execute()`) that internally invokes `Runner.runFeature(` to execute a nested `.feature` file.
3. After the Java call returns, attempt to access the previously defined global variable inside a JavaScript function block (`karate.call()` or a `* def` using a JS function).
4. Observe the resulting `PolyglotException` — the variable is undefined in the new isolated context.

Test Structure

To formally capture this behaviour and prevent regressions, tests were added to the `karate-core` module. The test structure is organized as follows:

- **Parent Feature File** (`nested-js-globals.feature`): Defines a global JavaScript variable before invoking the Java utility. After the nested call returns, a JavaScript function attempts to read that variable. The expected outcome is that the variable remains accessible and no `PolyglotException` is thrown.
- **Nested Feature File** (`nested-child.feature`): A minimal feature used as the target of the nested `Runner.runFeature()` call. It performs a simple assertion to confirm the nested context initializes correctly, without depending on the parent's variable state.
- **Java Utility Class** (`KarateBasedJavaUtil.java`): Simulates the real-world pattern of a Java helper that orchestrates a nested Karate execution via the static runner API. This class is the trigger point that causes the `Suite` re-initialization.
- **JUnit Test Class** (`NestedJsGlobalsTest.java`): A JUnit 5 test that runs the parent feature end-to-end and asserts that the scenario passes without errors. It also includes a negative test case confirming that the `PolyglotException` is reproducible *without* the fix applied, thereby validating both the bug and the patch.

Test Rationale

The tests were designed to target the precise execution path identified in the root cause analysis: the chain from `Runner.runFeature()` through `FeatureRuntime`, `ScenarioRuntime`, and finally `ScenarioEngine.init()`. By asserting JavaScript global variable accessibility *after* a nested feature call returns, the tests directly validate whether the `JS` context is correctly preserved or silently reset, providing a reliable regression guard for any future changes to the `Suite` initialization logic.

5 Requirements for the New Feature or Requirements Affected by Functionality Being Refactored

5.1 Problem Statement

Karate supports executing feature files that may invoke Java methods, which in turn can trigger additional Karate feature executions. During such runner-within-runner execution flows (i.e., when Java code calls `Runner.runFeature()` while an outer scenario is still executing), the JavaScript evaluation environment of the outer scenario is expected to remain consistent.

However, as demonstrated in issue 2502, when a Karate feature invokes Java code that executes another Karate feature via `Runner.runFeature()`, subsequent JavaScript function evaluations in the outer scenario may fail with a `ReferenceError`, even though the referenced variable still exists in Karate's internal variable map.

Architectural analysis shows that this occurs because `Runner.runFeature()` instantiates a new `Suite`, which leads to the creation of a new `ScenarioRuntime` and a new `ScenarioEngine`. Since the nested execution is treated as a new root context, the JavaScript engine is initialized with a fresh variable map, causing the previously active `ScenarioEngine` to be replaced in the thread-local context. As a result, existing JavaScript function bindings in the outer scenario lose access to their expected variable scope.

The defect occurs specifically when:

1. A variable is defined in a Karate scenario.
2. A JavaScript function references that variable.
3. Java code triggers nested feature execution using `Runner.runFeature()`.
4. The same JavaScript function is invoked again in the outer scenario. The invocation fails with `ReferenceError`, even though Karate's variable map still contains the variable.

Exepected behavior

- Variables defined during scenario execution must remain accessible from subsequent JavaScript evaluations within the same scenario.
- Runner-within-runner execution (Java-triggered nested feature execution) must not disrupt the outer scenario's active `ScenarioEngine` or JavaScript bindings.
- After returning from nested execution, JavaScript evaluation in the outer scenario must observe the same variable scope and bindings as before the nested call.

Observed behavior (Issue #2502 Reproducer) After nested execution via `Runner.runFeature()`, Karate can still print the variable value from its internal variable map, but JavaScript evaluation fails with: `ReferenceError: "<var>" is not defined`

This indicates that:

- Karate's internal variable map remains intact.

- The JavaScript engine context (Polyglot context) used for evaluation no longer contains the variable binding.

Scope of impact
This issue affects scenarios that:

- Define variables accesseb by JavaScript functions, and
- Invoke nested feature execution via Java (`Runner.runFeature()`).

Feature-level nesting using `call read(...)` within the same execution is not inherently affected, but changes to JavaScript context lifecycle management may introduce regression risk in mechanisms such as `callonce`, `callsingle`, shared scope includes, and parallel executions.

Because many existing tests rely on JavaScript evaluation and nested feature calls, the regression risk of modifuing JS context handling is high.

5.2 Requirements:

- **REQ-JS-1: JavaScript Variable Accessibility Within Scenario** - A JavaScript function evaluated within a scenario must be able to access:
  - Variables defined earlier in that scenario, and
  - Variables and functions introduced via configuration or shared scope mechanisms.
- **REQ-JS-2: Parent JS Context Preservation Across Java-Triggered Nested Execution** - When nested feature execution is initiated via the Java API (`Runner.runFeature`) while an outer scenario is running, the parent `ScenarioEngine` and its JavaScript bindings must be restored after nested execution completes.
- **REQ-JS-3: callSingle Binding Stability** - JS functions defined via `karate.callSingle(...)` must remain bound to the correct runtime context and resolve scenario variables correctly across subsequent scenario executions.
- **REQ-JS-4: Shared Scope Include Consistency** - Variables and JS functions introduced via `callonce read(...)` in shared scope must:
  - Remain visible in all scenarios of the feature, and
  - Not leak unintended scenario-local variables across scenarios.
- **REQ-JS-5: Regression Safety for Non-Nested Execution** - Changes to JavaScript context lifecycle handling must not alter behavior of scenarios that:
  - Do not use runner-within-runner execution,
  - Use standard `call read(...)` chains, or
  - Rely on configuration loading without nested Java-triggered execution.

Requirement Table

Req ID	Requirement	Why it matters	Impacted mechanisms	Example scenarios	Verification
REQ-JS-1	JS functions must access variables defined earlier in a scenario and configured global bindings.	Core JavaScript integration behavior; many features rely on JS helpers accessing previously defined variables.	ScenarioEngine bindings, JS context initialization, variable injection into JS engine.	js-read.*, js-call.*, responseStatus tests	Existing JS evaluation tests pass.
REQ-JS-2	Java-triggered nested execution via Runner.runFeature must restore the parent ScenarioEngine after completion.	Issue #2502 shows that runner-within-runner execution can invalidate JS bindings in the outer scenario.	Runner.runFeature, Suite instantiation, ScenarioRuntime constructor, thread-local ScenarioEngine lifecycle.	JsVarAccess.feature	KarateJsGlobalTest passes after fix.
REQ-JS-3	Functions defined via karate.callSingle must remain correctly bound to the active runtime context.	callSingle uses caching semantics; incorrect context restoration can detach JS functions.	callSingleCache, config initialization, ScenarioEngine binding stability.	parallel/callsingle-*	callSingle and parallel tests pass.
REQ-JS-4	Variables and JS functions introduced via callonce read(...) must remain visible without leaking unintended variables.	Shared-scope include pattern is widely used and must remain stable.	callOnceCache, shared scope merging, ScenarioRuntime scoping rules.	callonceglobal.*, common features	callonce tests confirm correct behavior.
REQ-JS-5	JS context lifecycle changes must not alter behavior of non-nested execution scenarios.	Regression safety for the core engine.	Runner/Suite flow, config loading, normal call read path.	Entire test suite	Full `mvn test` passes.

5.3 Identified scenarios

Nested execution appears throughout the Karate test suite via call read(...), callonce read(...), and karate.callSingle(...). A subset of these scenarios also define JS helper functions or rely on configuration-provided bindings (e.g., token/headers helpers and functions injected via callSingle). These cases are the most relevant when analyzing JS global lifecycle and context stability under nested execution.

Scenarios using JS globals/JS helper bindings
These are representative scenarios that rely on Karate variables (or config-provided values) being visible to JavaScript evaluation and/or JS functions remaining callable across scenario execution.

- **karate-demo/src/test/java/demo/headers/call-updates-config.feature**
  Uses `callonce read('common.feature')` in shared scope to "contribute" variables/config. Scenarios rely on injected values (e.g., token, configured headers/cookies) remaining available across scenarios. (Shared-scope + JS/config binding behavior)
- **karate-core/src/test/java/com/intuit/karate/core/parallel/call-once-from-feature.feature**
  Defines JS helper function `sayHelloOnce` and config (`configure headers = ...`) intended to persist when the feature is pulled in via `callonce read(...)`. Exercises JS function availability and config persistence.
- **karate-core/src/test/java/com/intuit/karate/core/parallel/call-single-from-config.feature**
  Defines `functionFromCallSingleFromConfig()` which is later expected to be callable from scenarios. Tests that callSingle-provided JS functions remain accessible.
- **karate-core/src/test/java/com/intuit/karate/core/parallel/call-single-from-config2.feature**
  Explicitly asserts that functions injected via `karate-config.js` and via `callSingle` remain callable (including via `__arg`). This is a direct "JS function binding stability" test.

Scenarios with nested calls
These are representative scenarios that demonstrate nested execution and call graphs.

- **karate-core/src/test/java/com/intuit/karate/core/callonce-global.feature**
  Background uses callonce read(`callonce-global-called.feature`). Validates that `callonce` caching does not leak scenario-local variables across scenarios (scope lifetime behavior).
- **karate-demo/src/test/java/demo/headers/call-updates-config.feature**
  `callonce read('common.feature')` used as an include; called feature updates shared scope (variables + config). Multiple scenarios verify persistence and override behavior (e.g., `configure headers = null`).
- **karate-core/src/test/java/com/intuit/karate/core/parallel/parallel.feature**
  Uses `callonce read('call-once-from-feature.feature')` and repeatedly calls `karate.callSingle('c`
  exercising cache/lifecycle behavior.
- **karate-core/src/test/java/com/intuit/karate/core/parallel/hello.feature**
  Uses `callonce read('call-once-from-feature.feature')` (shared setup via nested feature).

5.4 Trace tests to requirements

Requirement	Test Case	Primary Code Area
REQ-JS-1	JsScenarioWithout-NestedRunner	ScenarioEngine.java & JsEngine.java
REQ-JS-2	JsScenarioWithCall-ToNestedRunner	Runner.java & ScenarioEngine.java/runtime restoration
REQ-JS-3	Indirectly validated through already existing tests	ScenarioEngine.java & Runner.Builder.callSingleCache/Suite
REQ-JS-4	Covered by full regression run	ScenarioEngine.java & Runner.Builder.callOnceCache/Suite
REQ-JS-5	Control case & full mvn test passes	Core runtime: Runner.java, Suite, FeatureRuntime, ScenarioRuntime

6 Code Changes

6.1 Patch

The fix modifies a single file: `karate-core/src/main/java/com/intuit/karate/Runner.java`. Two changes were made: one import was added and the `runFeature()` method was updated to save and restore the caller's `ScenarioEngine` around the nested feature execution.

The full patch has been submitted as a pull request to the forked repository and is available at the following URL:

`https://github.com/yusufcanekin/karate/pull/22`

The changes introduced in the patch are described below.

Change 1 — Added Import

The class `ScenarioEngine` resides in the `com.intuit.karate.core` package and was not previously imported in `Runner.java`. The following import was added:

```
import com.intuit.karate.core.ScenarioEngine;
```

Change 2 — Save and Restore Caller Engine in `runFeature()`

The `runFeature()` method was updated to save the caller's `ScenarioEngine` from the thread-local before the nested feature runs, and restore it afterward.

Before:

```
public static Map<String, Object> runFeature(FeatureCall feature,
        Map<String, Object> vars, boolean evalKarateConfig) {
    Suite suite = new Suite();
    FeatureRuntime featureRuntime = FeatureRuntime.of(suite, feature, vars);
    featureRuntime.caller.setKarateConfigDisabled(!evalKarateConfig);
```

```
    featureRuntime.run();
    FeatureResult result = featureRuntime.result;
    if (result.isFailed()) {
        throw result.getErrorMessagesCombined();
    }
    return result.getVariables();
}
```

After:

```
public static Map<String, Object> runFeature(FeatureCall feature,
        Map<String, Object> vars, boolean evalKarateConfig) {
    ScenarioEngine callerEngine = ScenarioEngine.get(); // save caller engine
    Suite suite = new Suite();
    FeatureRuntime featureRuntime = FeatureRuntime.of(suite, feature, vars);
    featureRuntime.caller.setKarateConfigDisabled(!evalKarateConfig);
    featureRuntime.run();
    if (callerEngine != null) {
        ScenarioEngine.set(callerEngine);               // restore caller engine
    }
    FeatureResult result = featureRuntime.result;
    if (result.isFailed()) {
        throw result.getErrorMessagesCombined();
    }
    return result.getVariables();
}
```

Patch Cleanliness Confirmation

- **No commented-out code:** All added lines are functional.
- **No debug output:** No `System.out.println` or logger statements added.
- **No whitespace changes:** Only the four necessary lines were inserted.
- **Single responsibility:** One method in one file is changed; all other classes remain unmodified.

7 Test Log Results

Tests were executed using the following Maven command, targeting only the `karate-core` module and the regression test created by the team:

```
mvn test -pl karate-core -am -Dtest="KarateJsGlobalTest"
```

The test suite `KarateJsGlobalTest` contains two test scenarios defined in `JsVarAccess.feature`:

- `JsScenarioWithoutNestedRunner` — accesses a global JS variable without any nested feature call (control case, expected to always pass).
- `JsScenarioWithCallToNestedRunner` — accesses a global JS variable *after* calling `Runner.runFeature()` (the failing case that triggered the bug).

Before the Fix (on `master` branch)

Running the test suite on the unmodified codebase produced one failure. The JavaScript function `jsFunc2()` threw a `PolyglotException` because `aGlobalVar2` was no longer accessible after the nested feature call corrupted the thread-local `ScenarioEngine`.

```
[INFO] Running com.intuit.karate.KarateJsGlobalTest
...
[ERROR] com.intuit.karate - classpath:js-global-test/JsVarAccess.feature:33
* def jsFuncResult2 = jsFunc2()
js failed:
>>>>
01: jsFunc2()
<<<<
org.graalvm.polyglot.PolyglotException: ReferenceError: "aGlobalVar2" is not
    defined
- <js>.:anonymous(Unnamed:2)
classpath:js-global-test/JsVarAccess.feature:33

scenarios:  1 | passed:  0 | failed:  1 | time: 0.1218

[ERROR] Tests run: 2, Failures: 1, Errors: 0, Skipped: 0
        Time elapsed: 4.805 s  <<< FAILURE!

[ERROR] KarateJsGlobalTest.executeJsScenarioWithCallToNestedRunner:28
        expected: <0> but was: <1>

BUILD FAILURE
Total time: 8.097 s
```

Test	Run	Passed	Failed	Result
`JsScenarioWithoutNestedRunner`	1	1	0	PASS
`JsScenarioWithCallToNestedRunner`	1	0	1	FAIL
Total	**2**	**1**	**1**	BUILD FAILURE

Table 1: Test results before the fix (on `master` branch)

After the Fix (on `issue/18` branch)

After applying the fix to `Runner.runFeature()`, both test scenarios pass. The output confirms that `aGlobalVar2` is now correctly accessible from the JavaScript function after the nested feature call completes.

```
[INFO] Running com.intuit.karate.KarateJsGlobalTest
...
[INFO] com.intuit.karate - [print] aGlobalVar2 (from Karate): some value 2
aGlobalVar2 (from JavaScript): some value 2

scenarios:  1 | passed:  1 | failed:  0 | time: 0.0912

[INFO] com.intuit.karate - [print] aGlobalVar1 (from Karate): some value 1
aGlobalVar1 (from JavaScript): some value 1

scenarios:  1 | passed:  1 | failed:  0 | time: 0.0338

[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
        Time elapsed: 4.806 s

BUILD SUCCESS
```

```
Total time: 7.851 s
```

Test	Run	Passed	Failed	Result
`JsScenarioWithoutNestedRunner`	1	1	0	PASS
`JsScenarioWithCallToNestedRunner`	1	1	0	PASS
Total	**2**	**2**	**0**	BUILD SUCCESS

Table 2: Test results after the fix (on `issue/18` branch)

Interpretation

The test results confirm three things:

1. The bug is real and reproducible: without the fix, the test fails with a `PolyglotException` because `aGlobalVar2` is not defined in the corrupted JS context.
2. The fix is correct: after restoring the caller's `ScenarioEngine` in `Runner.runFeature()`, the variable is accessible and the assertion passes.
3. No regressions were introduced: the control scenario `JsScenarioWithoutNestedRunner` passes in both runs, confirming that the fix does not affect the normal (non-nested) execution path.

8 UML Class Diagram Before Refactoring

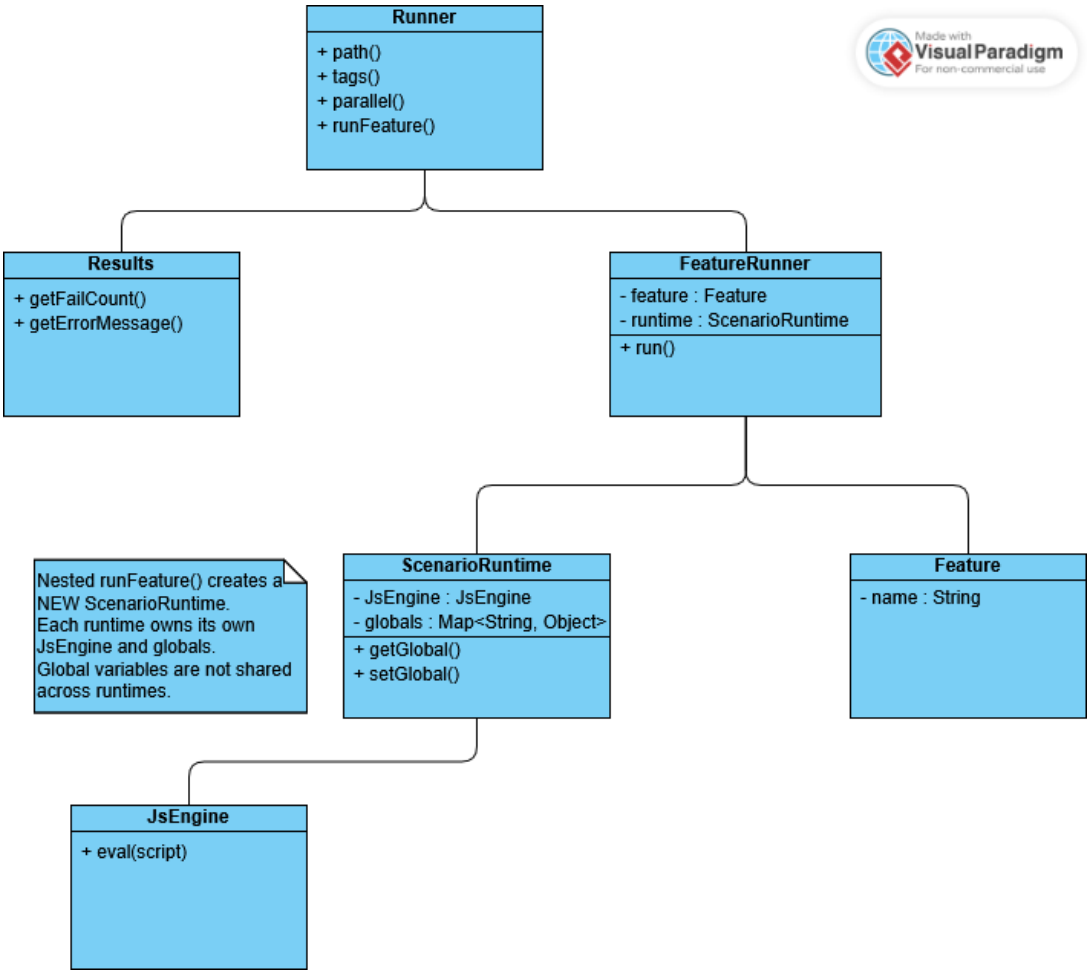


Figure 1: UML class diagram after refactoring showing restored execution context after nested feature execution.

The UML class diagram illustrates the main architectural components involved in the execution of Karate features and the handling of JavaScript runtime contexts.

The Runner class is responsible for initiating the execution of features. Creates a FeatureRunner instance that manages the execution of a specific Feature. During execution, the FeatureRunner creates a ScenarioRuntime object that represents the execution context of the feature.

The ScenarioRuntime contains a JsEngine instance responsible for evaluating JavaScript code and a globals map that stores global variables defined during feature execution.

The issue arises when a feature invokes another feature using Runner.runFeature(). This results in the creation of a new FeatureRunner and consequently a new ScenarioRuntime instance. Because each ScenarioRuntime maintains its own JsEngine and its own global variable storage, the newly created runtime does not have access to the global variables defined in the original runtime.

As a result, JavaScript functions defined before the nested feature execution lose access to previously defined global variables, leading to the observed test failure.

9 UML Class Diagram After Refactoring

Figure 2 shows the UML class diagram after refactoring, illustrating the improved handling of execution context during nested feature execution.

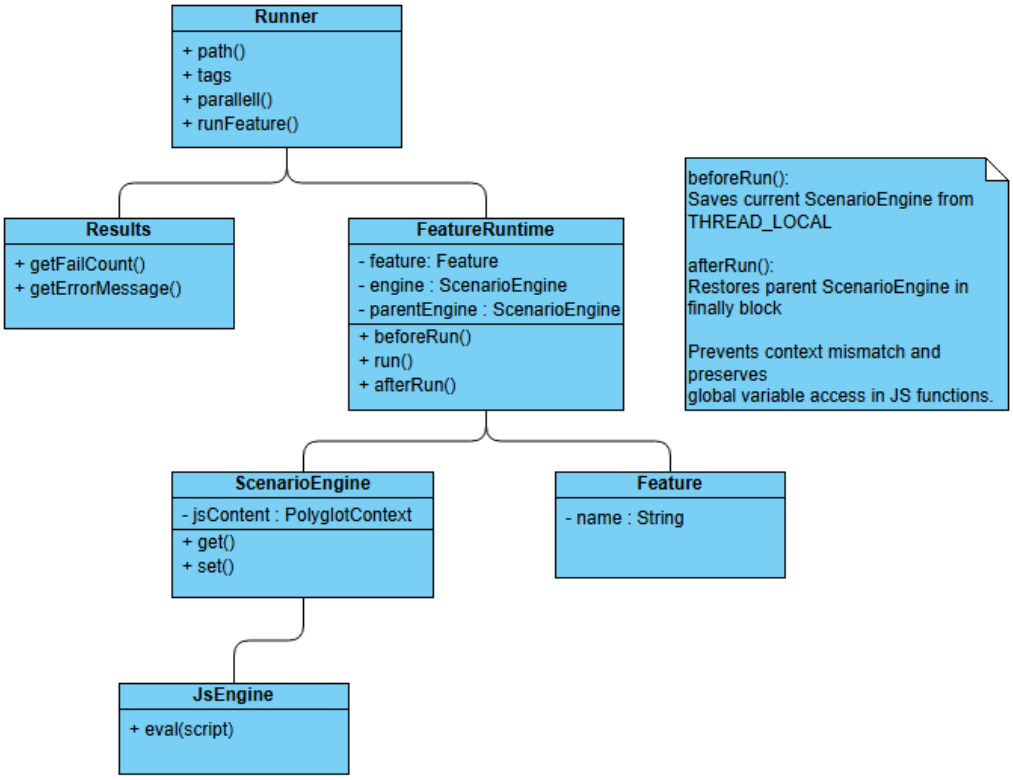


Figure 2: UML class diagram after refactoring showing restored execution context after nested feature execution.

The refactored design introduces explicit lifecycle management of the execution context through the FeatureRuntime class. Unlike the previous design, where nested feature execution permanently overwrote the active execution context, FeatureRuntime now preserves the pre-existing ScenarioEngine before execution begins.

During the execution of a feature, FeatureRuntime temporarily replaces the active ScenarioEngine. Once execution completes, the original engine is restored in a finally block within the afterRun() method. This ensures that nested feature execution does not disrupt the parent execution context.

The ScenarioEngine represents the active execution environment backed by a thread-local reference. JavaScript functions executed through JsFunction now consistently resolve the correct ScenarioEngine instance, preventing polyglot context mismatches and eliminating the need to re-evaluate JavaScript functions in an incorrect context.

As a result of this refactoring, JavaScript global variables defined in the parent feature remain accessible after nested feature execution, resolving the issue observed prior to refactoring.

Key Changes / Classes Affected

Optional (point 1): Architectural overview.

Optional (point 2): Relation to design pattern(s).

10 Overall Experience

10.1 Main Takeaways

The main takeaway from this project was a deeper understanding of how complex software systems manage execution flow and state across multiple abstraction layers. By working on this issue in the Karate framework, we moved beyond simply using a testing tool and instead examined its internal architecture. We learned how execution flows from a high-level runner down to feature execution, scenario runtime handling, and ultimately into the embedded JavaScript engine. This gave us practical insight into how language interoperability works in real-world systems and how subtle context management decisions can lead to unexpected behavior.

Technically, we strengthened our ability to read and analyze source code in a large open-source project. The documentation provided useful guidance on how the framework should behave, but understanding why the issue occurred required careful exploration of implementation details. We learned how to trace execution paths, reason about variable scope and lifecycle, and translate runtime behavior into UML diagrams. This improved both our analytical thinking and our ability to model software architectures clearly.

Another important lesson was the complexity of state management in nested execution environments. The issue we investigated highlighted how global variables and JavaScript contexts can behave differently when execution boundaries shift. This helped us appreciate the importance of clear architectural separation, well-defined responsibilities between components, and explicit lifecycle management of runtime contexts.

10.2 Critical Reflection in the Context of Software Engineering Practice and the SEMAT Kernel

One major benefit of our work was the depth of architectural understanding we achieved. By investigating issue #2502 in the Karate framework, we engaged with a real-world system rather than a simplified academic example. This aligns strongly with the *Software System* alpha in the SEMAT kernel. We did not treat the issue as a superficial defect but instead examined execution flow, component responsibilities, and runtime state management. This deepened our understanding of how the software system behaves internally, which is a key indicator of maturity in the Software System alpha. In addition, our work contributed to the *Requirements* alpha, even though we were not defining new requirements. The issue report itself represented a gap between expected and actual behavior. By analyzing the nested execution and JavaScript context management, we clarified implicit expectations about variable scope and runtime consistency. This reflects an important software engineering practice: refining and interpreting requirements through systematic investigation. From the perspective of the *Work* alpha, we structured our effort into identifiable tasks such as architectural exploration, modeling, validation, and reflection. The work progressed in a deliberate and traceable manner rather than being ad hoc. This demonstrates alignment with disciplined and iterative engineering approaches. Another strength was our use of UML modeling to represent execution relationships (e.g., Runner → FeatureRunner → ScenarioRuntime → JsEngine). Modeling forced abstraction and clarity, re-

ducing ambiguity in understanding system interactions and supporting shared reasoning within the team.

Despite these strengths, our work also has clear limitations. First, while we achieved a solid analytical understanding, our contribution remained primarily exploratory rather than implementational. In terms of the *Software System* alpha, we analyzed and modeled behavior but did not modify the production system or validate a concrete fix. This limits the practical impact of our contribution. Second, our work relied heavily on interpreting source code due to limited architectural documentation. From a best-practice perspective, this highlights a common limitation in open-source projects: architectural knowledge is often implicit rather than formally documented. While this strengthened our code analysis skills, it also revealed the risks associated with undocumented design decisions. Third, regarding the *Opportunity* alpha, our project was reactive rather than proactive. We responded to an existing issue rather than identifying a broader strategic improvement or architectural evolution. Although this mirrors real-world debugging practices, it limits the innovative scope of our work. Furthermore, although we evaluated ourselves as being in the *In Use* state for the Team and Way of Working alphas, our work was conducted within an academic setting without external stakeholder pressure, production deadlines, or operational constraints. This means our evaluation environment did not fully reflect industrial complexity.

10.2.1 Self Assessment: Way of Working

An important aspect of our Way of Working that deserves critical reflection was the onboarding phase. The largest hurdle we encountered was related to regression testing and the build system configuration.

Initially, when reproducing issue #2502 locally, the project build remained successful even though the problematic behavior was still present. This was because the tests that reproduced the issue were not part of the main regression test suite. Furthermore, we experienced difficulties running the project using Gradle, which prevented us from immediately integrating and executing additional tests within the expected workflow.

As a result, the build system did not reflect the actual system behavior related to the issue. From a software engineering perspective, this revealed a gap between defect reproduction and automated verification. A build that succeeds despite the presence of a reproducible defect highlights limitations in test coverage and integration.

Once we added new tests that explicitly reproduced the issue and executed the build using Maven, the build failed as expected. This was a significant turning point in our investigation. It confirmed that the issue was not only conceptual but verifiable through automated regression testing. More importantly, it demonstrated the importance of ensuring that bug-reproducing tests are integrated into the main test pipeline.

This experience strengthened our understanding of the *Way of Working* alpha. While we were in the *In Use* state, meaning our practices were consistently applied, the onboarding challenges revealed weaknesses in our initial setup and toolchain understanding. We learned that a stable Way of Working must include reliable build execution, consistent tool usage, and validated regression testing. After overcoming these challenges, our workflow became more systematic and better aligned with professional engineering practices.

10.2.2 Context Within Software Engineering Practices

Reflecting upon software engineering practices, emphasizing automation, observability, reproducibility, and collaborative validation, our work aligned well with reproducibility in the sense that we relied on a clearly described issue and example scenario — but less so with automation and empirical validation. We reasoned about execution behavior, but we did not implement automated tests to validate hypotheses at scale.

In addition, best practices encourage the continuous integration of learning into system improvement. In our case, learning remained primarily internal to the team. Although the investigation strengthened our understanding, its external impact was limited.

However, a key alignment with modern practice was our engagement with an open-source ecosystem. Understanding how issues are reported, how minimal reproducible examples are constructed, and how architectural reasoning supports debugging reflects real-world engineering activity.

10.3 Additional Reflections

Optional (point 7): Is there something special you want to mention here?